

1.2_JH_xi_xidot

March 13, 2026

```
[79]: import torch
import numpy as np
import torch.nn as nn
from tqdm import trange
from pathlib import Path
import torch.optim as optim
import matplotlib.pyplot as plt
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import StandardScaler, MinMaxScaler
import importlib
from torchinfo import summary
import time
```

```
import sys
sys.path.append('.')
import plot_settings
plot_settings.apply()
import sim_PyBaMM
importlib.reload(sim_PyBaMM);
import exp_pybamm_ver4
importlib.reload(exp_pybamm_ver4);
import SPM_eqs
importlib.reload(SPM_eqs);
import Euler_exp_log
importlib.reload(Euler_exp_log);
```

```
[127]: # --- Simulation config ---
T_HORIZON = 3600.0          # [s] total simulation horizon
T = 7200                    # number of time steps
DT = T_HORIZON / (T - 1)   # fixed for all sims
t_fixed = np.linspace(0, T_HORIZON, T)   # fixed 'physical' grid
```

```
[133]: # --- Data generation ---

def time_query(sol, t_fixed):
    t_end = sol["Time [s]"].entries[-1]
```

```

    valid = t_fixed <= t_end # boolean, length T # Valid mask: where
    ↪simulation was still running
    t_query = np.clip(t_fixed, 0, t_end) # Observe only up to t_end, fill
    ↪rest with NaN or last value
    return t_query, valid

def observe_states(t_query, sol, model):
    cn = sol.observe(model.variables["X-averaged negative particle
    ↪concentration [mol.m-3]"])(t_query)[-1, :]
    cp = sol.observe(model.variables["X-averaged positive particle
    ↪concentration [mol.m-3]"])(t_query)[-1, :]
    phin = sol.observe(model.variables["X-averaged negative electrode potential
    ↪[V]"])(t_query)
    phip = sol.observe(model.variables["X-averaged positive electrode potential
    ↪[V]"])(t_query)
    phie = sol.observe(model.variables["X-averaged electrolyte potential
    ↪[V]"])(t_query)
    I = sol.observe(model.variables["Current [A]"])(t_query)
    V = sol.observe(model.variables["Terminal voltage [V]"])(t_query)
    return cn, cp, phin, phip, phie, I, V

def get_vals_DC(I0):
    sol, model, param = sim_PyBaMM.simulate_DC1(I0, T, T_HORIZON) # simulate
    ↪with t_eval = np.linspace(0, t_horizon, T)
    t_query, valid = time_query(sol, t_fixed)
    cn, cp, phin, phip, phie, I, V = observe_states(t_query, sol, model)
    return V, I, cn, cp, phin, phip, phie, valid

def get_vals_pulse():
    sol, model, t_eval, param = exp_pybamm_ver4.generate_data(T_HORIZON, DT,
    ↪model='DFN', RNG=None) # simulate with t_eval = np.linspace(0, t_horizon,
    ↪T)
    t_query, valid = time_query(sol, t_fixed)
    cn, cp, phin, phip, phie, I, V = observe_states(t_query, sol, model)
    return V, I, cn, cp, phin, phip, phie, valid

def get_vals_GRF():
    sol, model, param = sim_PyBaMM.simulate_GRF(T, T_HORIZON) # simulate with
    ↪t_eval = np.linspace(0, t_horizon, T)
    t_query, valid = time_query(sol, t_fixed)
    cn, cp, phin, phip, phie, I, V = observe_states(t_query, sol, model)
    return V, I, cn, cp, phin, phip, phie, valid

def gen_data(B, currents='DC', I_low=0.25, I_high=1.0, seed=None):
    ''' Generates B trajectories. Returns arrays of shape (B, T, 1) '''

```

```

rng = np.random.default_rng(seed)

arrays = {k: np.zeros((B, T, 1)) for k in ['I', 'dI', 'V', 'cn', 'cn_dot',
↪ 'cp', 'cp_dot', 'phin', 'phin_dot', 'phip', 'phip_dot', 'phie', 'phie_dot']}
valid = np.zeros((B, T), dtype=bool)

for b in trange(B):
    if currents == 'DC':
        V_b, I_b, cn_b, cp_b, phin_b, phip_b, phie_b, valid_b =
↪ get_vals_DC(float(rng.uniform(I_low, I_high)))
        elif currents == 'pulse':
            V_b, I_b, cn_b, cp_b, phin_b, phip_b, phie_b, valid_b =
↪ get_vals_pulse()
            elif currents == 'GRF':
                V_b, I_b, cn_b, cp_b, phin_b, phip_b, phie_b, valid_b = get_vals_GRF()

        cn_dot_b = np.zeros_like(cn_b)
        cp_dot_b = np.zeros_like(cp_b)
        cn_dot_b[:-1] = np.diff(cn_b) / DT # Derivative w.r.t. time using
↪ backward difference
        cp_dot_b[:-1] = np.diff(cp_b) / DT
        cn_dot_b[-1] = cn_dot_b[-2] # Pad last entry with second to
↪ last value
        cp_dot_b[-1] = cp_dot_b[-2]

        phin_dot_b = np.zeros_like(phin_b)
        phip_dot_b = np.zeros_like(phip_b)
        phie_dot_b = np.zeros_like(phie_b)
        phin_dot_b[:-1] = np.diff(phin_b) / DT
        phip_dot_b[:-1] = np.diff(phip_b) / DT
        phie_dot_b[:-1] = np.diff(phie_b) / DT
        phin_dot_b[-1] = phin_dot_b[-2]
        phip_dot_b[-1] = phip_dot_b[-2]
        phie_dot_b[-1] = phie_dot_b[-2]

        for key, val in zip(['I', 'dI', 'V', 'cn', 'cn_dot', 'cp',
↪ 'cp_dot', 'phin', 'phin_dot', 'phip', 'phip_dot', 'phie', 'phie_dot'], [V_b, I_b,
↪ cn_b, cp_b, cn_dot_b, cp_dot_b, phin_b, phip_b, phie_b, phin_dot_b,
↪ phip_dot_b, phie_dot_b]):
            arrays[key][b, :, 0] = val

        arrays['dI'][b, 1:, 0] = np.diff(I_b)
        arrays['dI'][b, 0, 0] = 0.0 # dI[0] = 0
        valid[b, :] = valid_b

```

```

    return *[arrays[key] for key in ['I', 'dI', 'V', 'cn', 'cn_dot', 'cp',
    ↪ 'cp_dot', 'phin', 'phin_dot', 'hip', 'hip_dot', 'phie', 'phie_dot']] , valid

```

```

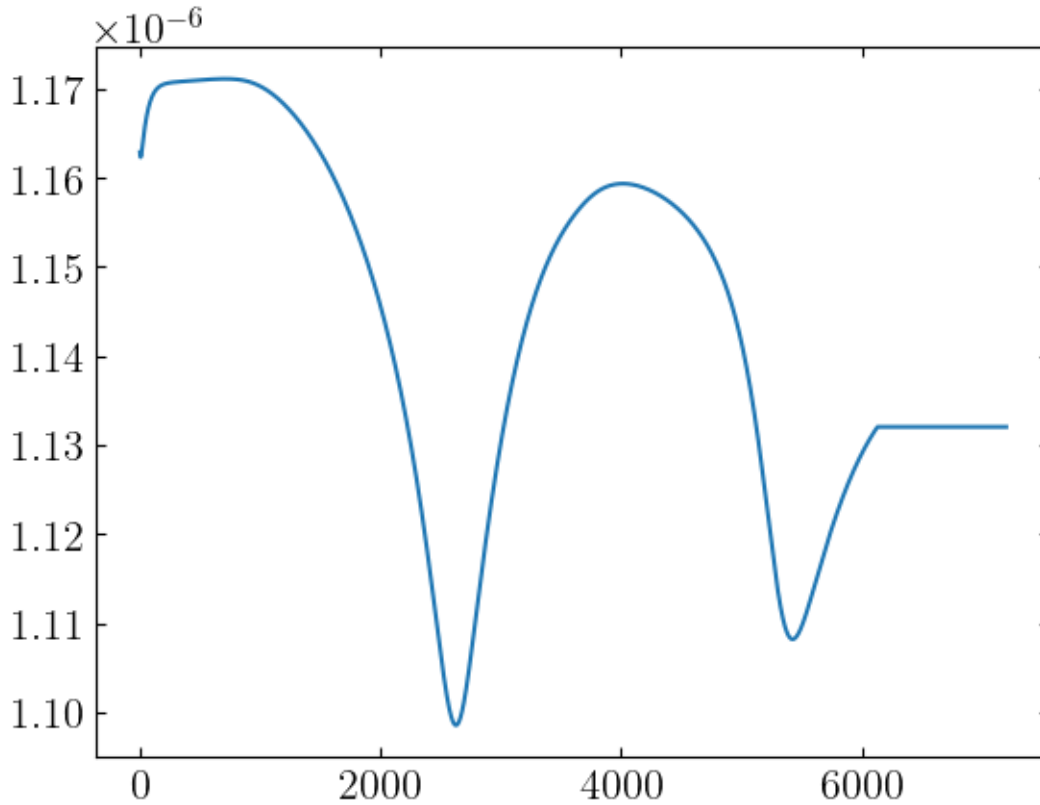
[134]: # --- Check ---
sol, model_bat, t_eval, param = exp_pybamm_ver4.generate_data(T_HORIZON, DT,
    ↪ model='DFN', RNG=None) # simulate with t_eval = np.linspace(0, t_horizon,
    ↪ T)
sol, model_bat, param = sim_PyBaMM.simulate_DC1(0.8, T, T_HORIZON)
t_query, valid = time_query(sol, t_fixed)

def observe_states_extra(t_query, sol, model):
    cn = sol.observe(model.variables["X-averaged negative particle
    ↪ concentration [mol.m-3]"])(t_query)[-1, :]
    cp = sol.observe(model.variables["X-averaged positive particle
    ↪ concentration [mol.m-3]"])(t_query)[-1, :]
    I = sol.observe(model.variables["Current [A]"])(t_query)
    V = sol.observe(model.variables["Terminal voltage [V]"])(t_query)
    Ohm_loss = sol.observe(model.variables["Negative electrode ohmic losses
    ↪ [V]"])(t_query)
    return cn, cp, I, V, Ohm_loss

cn, cp, I, V, Ohm_loss = observe_states_extra(t_query, sol, model_bat)
plt.plot(Ohm_loss[0, :]);
print(param['Contact resistance [Ohm]'], param['Current function [A]'])

```

10.0 0.8



```
[135]: # --- Save data ---
#B = 200
#I, dI, V, cn, cn_dot, cp, cp_dot, phin,phin_dot,phip,phip_dot,phie,phie_dot,
    ↪ valid = gen_data(B, currents='pulse', I_low=0.25, I_high=1.0, seed=42)
#np.savez('data_phi_c_pulse.npz', I=I, dI=dI, V=V, cn=cn, cn_dot=cn_dot, cp=cp,
    ↪ cp_dot=cp_dot, phin=phin, phin_dot=phin_dot, phip=phip, phip_dot=phip_dot,
    ↪ phie=phie, phie_dot=phie_dot, valid=valid)
```

100%| | 200/200 [02:33<00:00, 1.30it/s]

```
[138]: # --- Load data ---
def load_data(path):
    d = np.load(path)
    return d['I'], d['dI'], d['V'], d['cn'], d['cn_dot'], d['cp'], d['cp_dot'],
    ↪ d['phin'], d['phin_dot'], d['phip'], d['phip_dot'], d['phie'],
    ↪ d['phie_dot'], d['valid']

# I, dI, V, cn, cn_dot, cp, cp_dot, valid = load_data('Euler_data/data_c_GRFI.
    ↪ npz')
# I, dI, V, cn, cn_dot, cp, cp_dot, valid = load_data('t=5e3_GRF_data_c_dot.
    ↪ npz')
```

```
# I, dI, V, cn, cn_dot, cp, cp_dot, valid = load_data('data/data_c_pulse.npz')
I, dI, V, cn, cn_dot, cp, cp_dot, phin,phin_dot,phip,phip_dot,phie,phie_dot,
↪valid = load_data('data_phi_c_pulse.npz')
```

```
B = I.shape[0]
```

```
[145]: # --- Normalization ---
stats = {}
def compute_stats():
    for name, arr in zip(['I', 'dI', 'V', 'cn', 'cn_dot', 'cp', 'cp_dot',
↪'phin', 'phin_dot', 'phip', 'phip_dot', 'phie', 'phie_dot'],
                        [I, dI, V, cn, cn_dot, cp, cp_dot, phin, phip, phie,
↪phin_dot, phip_dot, phie_dot]):
        stats[name] = (float(arr.mean()), float(max(arr.std(), 1e-8))) # Avoid
↪zero std
    return stats

def transform(arr, name):
    if name in ['cn', 'cp']:
        # Normalize concentrations by their maximum values from the parameters
        if name == 'cn':
            mean = 0
            std = param['Maximum concentration in negative electrode [mol.m-3]']
        else:
            mean = 0
            std = param['Maximum concentration in positive electrode [mol.m-3]']

        if name in ['cn_dot', 'cp_dot']:
            mean = 0
            std = stats[name.replace('_dot', '')][1] / DT # Scale by time step for
↪derivatives

        if name not in ['cn', 'cp', 'cn_dot', 'cp_dot']:
            mean = 0
            std = 1
        mean, std = stats[name]
        return (arr - mean) / std

def inverse_transform(arr, name):
    if name in ['cn', 'cp']:
        # Denormalize concentrations by their maximum values from the parameters
        if name == 'cn':
            mean = 0
            std = param['Maximum concentration in negative electrode [mol.m-3]']
        else:
            mean = 0
            std = param['Maximum concentration in positive electrode [mol.m-3]']
```

```

    if name in ['cn_dot', 'cp_dot']:
        mean = 0
        std = 1 # stats[name.replace('_dot', '')][1] / DT # Scale by time step
    for derivatives
    if name not in ['cn', 'cp', 'cn_dot', 'cp_dot']:
        mean = 0
        std = 1
    mean, std = stats[name]
    return arr * std + mean

stats = compute_stats()

# Normalize
I_n = transform(I, 'I')
dI_n = transform(dI, 'dI')
cn_n = transform(cn, 'cn')
cp_n = transform(cp, 'cp')
cn_dot_n = transform(cn_dot, 'cn_dot')
cp_dot_n = transform(cp_dot, 'cp_dot')
phin_n = transform(phin, 'phin')
phip_n = transform(hip, 'hip')
phie_n = transform(phie, 'phie')
phin_dot_n = transform(phin_dot, 'phin_dot')
phip_dot_n = transform(hip_dot, 'hip_dot')
phie_dot_n = transform(phie_dot, 'phie_dot')
V_n = transform(V, 'V')

```

```

[ ]: # --- Split data ---
x = torch.tensor(np.concatenate([I_n, cn_n, cp_n, phin, phip, phie_n], axis=-1),
    dtype=torch.float32) # (B, T, 6)
y = torch.tensor(np.concatenate([cn_dot_n, cp_dot_n, phin_dot_n, phip_dot_n,
    phie_dot_n, V_n], axis=-1), dtype=torch.float32) # (B, T, 3)
mask = torch.tensor(valid, dtype=torch.float32)

split = int(0.8 * B)
def _split(arr):
    return arr[:split], arr[split:]

def _split_flat(arr):
    return arr[:split].flatten(0,1), arr[split:].flatten(0,1) # Same as
    reshape(B * T, C) # (B*T, 4)

# Flatten time dimension for training and split
x_train, x_test = _split_flat(x)
y_train, y_test = _split_flat(y)
mask_train, mask_test = _split_flat(mask)

```

```

# Un-normalized for plotting
cn_train,      cn_test      = _split(cn)
cp_train,      cp_test      = _split(cp)
cn_dot_train,  cn_dot_test  = _split(cn_dot)
cp_dot_train,  cp_dot_test  = _split(cp_dot)
I_train,       I_test       = _split(I)
dI_train,      dI_test      = _split(dI)
V_train,       V_test       = _split(V)
mask_train_plot, mask_test_plot = _split(valid)

print('Shapes:', x_train.shape, y_train.shape, mask_train.shape, cn_test.shape,
      ↪cp_test.shape, I_test.shape, V_test.shape)

batch_size_large = int(B*T/4)
train_loader_flat = DataLoader(TensorDataset(x_train, y_train, mask_train),
      ↪batch_size=batch_size_large, shuffle=True)

```

Shapes: torch.Size([1152000, 6]) torch.Size([1152000, 6]) torch.Size([1152000])
 (40, 7200, 1) (40, 7200, 1) (40, 7200, 1) (40, 7200, 1)

```

[150]: # --- Model ---
class Net(nn.Module):
    def __init__(self, input_size=6, hidden_size=128): # [I_t, cn_t, cp_t,
      ↪phin_t, phip_t, phie_t]
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, hidden_size),      # [I_t, cn_t, cp_t]
            nn.ReLU(),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 5 * 3 + 1)         # [G, m1, m2, l1,
      ↪l2, z1, z2]
        )

    def forward(self, x):
        x = self.net(x)
        return x

model = Net()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

```

```

[111]: # summary(model=model, input_size=(batch_size_small*T, 4))

```

```

[ ]: # --- Training ---
def masked_mse(pred, target, mask):
    err = (pred - target) ** 2 # (B*T, 2)
    m = mask.unsqueeze(-1)     # (B*T, 1)

```



```

return (err * m).sum() / (m.sum() * 2)

def state_evolver(pred, x):
    G, m1, m2,m3,m4,m5 ,l1, l2,l3,l4,l5 ,z1, z2,z3, z4, z5 = pred[:, 0], pred[:,
↪ 1], pred[:, 2], pred[:, 3], pred[:, 4], pred[:, 5], pred[:, 6], pred[:,
↪ 7], pred[:, 8], pred[:, 9], pred[:, 10], pred[:, 11], pred[:, 12], pred[:,
↪ 13], pred[:, 14], pred[:, 15] # (B*T,)

    v = torch.stack([m1, m2,m3,m4,m5], dim=-1) # (B*T, 2)
    M = v.unsqueeze(-1) * v.unsqueeze(-2) # (B*T, 2, 2)
    Z = torch.stack([z1, z2,z3,z4,z5], dim=-1) # (B*T, 2)

    dG_dxi = torch.autograd.grad(G, x, grad_outputs=torch.ones_like(G),
↪ create_graph=True)[0] # (B*T, 3)

    dG_dxi_col = dG_dxi[:, 1:].unsqueeze(-1) # Slice of dG/dI (B*T, 2) then
↪unsqueeze (B*T, 2, 1)

    term1 = (M @ dG_dxi_col).squeeze(-1) # (B*T, 2)

    I = x[:, 0:1] # (B*T, 1)
    term2 = I * Z # (B*T, 2)

    xi_dot = - term1 - term2

    # --- Gibbs free energy rate dG/dt ---
    # Correction factors
    # std_xi = torch.tensor([stats['cn'][1], stats['cp'][1]], dtype=torch.
↪float32) # (2,)
    # std_dot = torch.tensor([stats['cn_dot'][1], stats['cp_dot'][1]],
↪dtype=torch.float32) # (2,)
    # mean_dot = torch.tensor([stats['cn_dot'][0], stats['cp_dot'][0]],
↪dtype=torch.float32) # (2,)
    # std_I = torch.tensor(stats['I'][1], dtype=torch.float32)
    # mean_I = torch.tensor(stats['I'][0], dtype=torch.float32)
    # std_V = torch.tensor(stats['V'][1], dtype=torch.float32)
    # mean_V = torch.tensor(stats['V'][0], dtype=torch.float32)
    std_xi = torch.tensor([param['Maximum concentration in negative electrode',
↪[mol.m-3]'],
                           param['Maximum concentration in positive electrode',
↪[mol.m-3]'],
                           1, 1, 1], dtype=torch.float32) # (2,)
    std_dot = torch.tensor([1, 1, 1, 1, 1], dtype=torch.float32) # (2,)

```

```

I_phys = inverse_transform(I, 'I') # * std_I + mean_I # (B*T, 1)

dG_dxi_phys = dG_dxi_col.squeeze(-1) #/ std_xi #
↳ (B*T, 2) phys gradient
xi_dot_phys = xi_dot # (B*T, 2) phys rates

dG_dt = (dG_dxi_phys * xi_dot_phys).sum(dim=-1, keepdim=True) # (B*T, 1)
↳ 1) physical dG/dt

volt = (dG_dxi_col * Z.unsqueeze(-1)).sum(dim=1)
V_phys = torch.min(inverse_transform(volt, 'V'), torch.tensor(3.1)) # Clip
↳ to max voltage to avoid unphysical predictions

power = I_phys * V_phys
# power = (dG_dxi_phys * std_xi * term2).sum(dim=-1, keepdim=True)
diss = (dG_dxi_phys * (std_dot / std_xi) * term1).sum(dim=-1, keepdim=True)
return xi_dot, volt, dG_dt, Z, diss, power

def loss_xi(xi_dot_pred, target, mask):
    xi_loss = masked_mse(xi_dot_pred, target, mask)
    return xi_loss

def loss_volt(volt_pred, target, mask):
    v_loss = masked_mse(volt_pred, target, mask)
    return v_loss

def training(epochs=30):
    history = {"loss": [], "xi_loss": [], "v_loss": [], "time per ep": []}
    for ep in range(epochs):
        model.train()
        tot_ep_loss = 0.0
        tot_xi_loss = 0.0
        tot_volt_loss = 0.0
        start_t = time.perf_counter()

        for x_b, y_b, mask_b in train_loader_flat:
            x_b = x_b.float().requires_grad_(True) # ← must be BEFORE
            ↳ model(x_b)
            pred = model(x_b) # [B*T, 2]
            xi_dot_pred, volt, dG_dt, Z, term1, term2 = state_evolver(pred, x_b)
            xi_loss = loss_xi(xi_dot_pred, y_b[:, :5], mask_b)
            volt_loss = loss_volt(volt, y_b[:, 5:6], mask_b)
            # volt_loss = torch.tensor(0.0)

```

```

        ep_loss = xi_loss + volt_loss
        optimizer.zero_grad()
        ep_loss.backward()
        optimizer.step()

        tot_ep_loss += ep_loss.item()
        tot_xi_loss += xi_loss.item()
        tot_volt_loss += volt_loss.item()

    avg_loss = tot_ep_loss / len(train_loader_flat) # Average loss per
↪ batch
    avg_xi_loss = tot_xi_loss / len(train_loader_flat)
    avg_volt_loss = tot_volt_loss / len(train_loader_flat)
    end_t = time.perf_counter()

    history["loss"].append(avg_loss)
    history["xi_loss"].append(avg_xi_loss)
    history["v_loss"].append(avg_volt_loss)
    history["time per ep"].append(end_t - start_t)
    print(f"Epoch {ep+1} | loss {avg_loss:.3e} | xi loss {avg_xi_loss:.3e} |
↪ | volt loss {avg_volt_loss:.3e} | time {end_t - start_t:.2f}s")

    return history

```

```
[164]: history = training(epochs=20)
```

```
Epoch 1 | loss 2.839e+10 | xi loss 2.839e+10 | volt loss 6.469e-01 | time 13.60s
Epoch 2 | loss 2.839e+10 | xi loss 2.839e+10 | volt loss 1.568e+00 | time 15.83s
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[164], line 1
----> 1 history = training(epochs=20)

Cell In[163], line 77, in training(epochs)
    74 tot_volt_loss = 0.0
    75 start_t = time.perf_counter()
----> 77 for x_b, y_b, mask_b in train_loader_flat:
    78
↪     x_b = x_b.float().requires_grad_(True) # ← must be BEFORE model(x_b)
    79     pred = model(x_b) # [B*T, 2]

File ~/Library/CloudStorage/OneDrive-Personal/0_Skola/Thesis/MSc_thesis/program .
↪ venv/lib/python3.12/site-packages/torch/utils/data/dataloader.py:741, in
↪ _BaseDataLoaderIter.__next__(self)
    738 if self._sampler_iter is None:
    739     # TODO(https://github.com/pytorch/pytorch/issues/76750)

```

```

740     self._reset() # type: ignore[call-arg]
--> 741 data = self._next_data()
742 self._num_yielded += 1
743 if (
744     self._dataset_kind == _DatasetKind.Iterable
745     and self._IterableDataset_len_called is not None
746     and self._num_yielded > self._IterableDataset_len_called
747 ):

```

File ~/Library/CloudStorage/OneDrive-Personal/0_Skola/Thesis/MSc_thesis/program .
↳venv/lib/python3.12/site-packages/torch/utils/data/dataloader.py:801, in
↳_SingleProcessDataLoaderIter._next_data(self)

```

799 def _next_data(self):
800     index = self._next_index() # may raise StopIteration
--> 801     data = self._dataset_fetcher.fetch(index) # may raise StopIteration
802     if self._pin_memory:
803         data = _utils.pin_memory.pin_memory(data, self.
↳_pin_memory_device)

```

File ~/Library/CloudStorage/OneDrive-Personal/0_Skola/Thesis/MSc_thesis/program .
↳venv/lib/python3.12/site-packages/torch/utils/data/_utils/fetch.py:57, in
↳_MapDatasetFetcher.fetch(self, possibly_batched_index)

```

55 else:
56     data = self.dataset[possibly_batched_index]
---> 57 return self.collate_fn(data)

```

File ~/Library/CloudStorage/OneDrive-Personal/0_Skola/Thesis/MSc_thesis/program .
↳venv/lib/python3.12/site-packages/torch/utils/data/_utils/collate.py:401, in
↳default_collate(batch)

```

340 def default_collate(batch):
341     r"""
342     Take in a batch of data and put the elements within the batch into a
↳tensor with an additional outer dimension - batch size.
343
(...) 399     >>> default_collate(batch) # Handle `CustomType`
↳automatically
400     """
--> 401     return collate(batch, collate_fn_map=default_collate_fn_map)

```

File ~/Library/CloudStorage/OneDrive-Personal/0_Skola/Thesis/MSc_thesis/program .
↳venv/lib/python3.12/site-packages/torch/utils/data/_utils/collate.py:215, in
↳collate(batch, collate_fn_map)

```

209 transposed = list(
210     zip(*batch, strict=False)
211 ) # It may be accessed twice, so we use a list.
212 if isinstance(elem, tuple):
213     return [
--> 215         collate(samples, collate_fn_map=collate_fn_map)
216         for samples in transposed

```

```

217     ] # Backwards compatibility.
218 else:
219     try:

File ~/Library/CloudStorage/OneDrive-Personal/0_Skola/Thesis/MSc_thesis/program .
↳venv/lib/python3.12/site-packages/torch/utils/data/_utils/collate.py:155, in
↳collate(batch, collate_fn_map)
    153 if collate_fn_map is not None:
    154     if elem_type in collate_fn_map:
--> 155         return
↳collate_fn_map[elem_type](batch, collate_fn_map=collate_fn_map)
    157     for collate_type in collate_fn_map:
    158         if isinstance(elem, collate_type):

File ~/Library/CloudStorage/OneDrive-Personal/0_Skola/Thesis/MSc_thesis/program .
↳venv/lib/python3.12/site-packages/torch/utils/data/_utils/collate.py:275, in
↳collate_tensor_fn(batch, collate_fn_map)
    273     storage = elem._typed_storage()._new_shared(numel, device=elem.
↳device)
    274     out = elem.new(storage).resize_(len(batch), *list(elem.size()))
--> 275 return torch.stack(batch, 0, out=out)

KeyboardInterrupt:

```

```

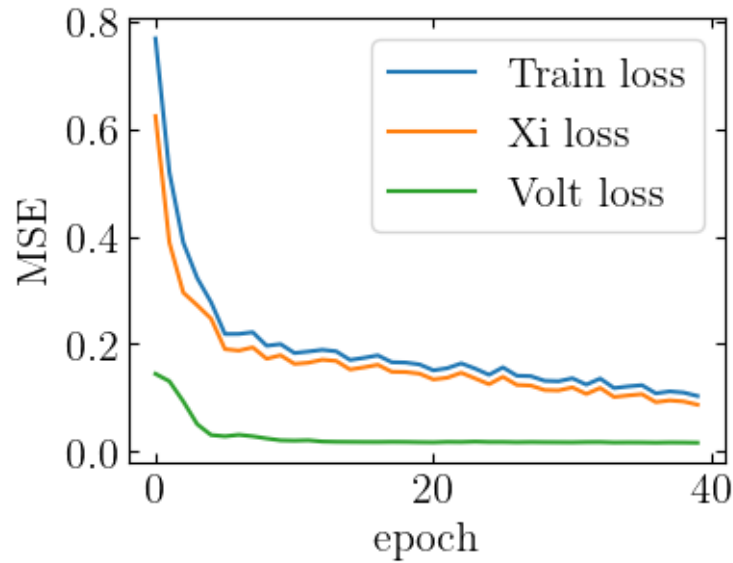
[114]: # Euler_exp_log.exp_log(model, history, name = 'Data dt=0.1', d_loss_type =
↳'mse')

```

```

[115]: fig, (ax1) = plt.subplots(1, 1, figsize=(4, 3))
ax1.plot(history["loss"], label="Train loss")
ax1.plot(history["xi_loss"], label="Xi loss")
ax1.plot(history["v_loss"], label="Volt loss")
ax1.set_xlabel("epoch")
ax1.set_ylabel("MSE")
ax1.legend();

```



```
[116]: # --- Euler rollout ---
def rollout(idx=0, dt_eval=DT, on_train=False):
    if on_train:
        cn_true = cn_train
        cp_true = cp_train
        I_true = I_train
    else:
        cn_true = cn_test
        cp_true = cp_test
        I_true = I_test

    model.eval()

    # Original time grid (training resolution)
    t_train = np.arange(T) * DT

    # Interpolate I onto the new dt grid
    def I_t(t):
        return float(np.interp(t, t_train, I_true[idx, :, 0])) # Value at time_
    ↪ t

    # Pre-allocate generously
    max_steps = int(T_HORIZON / dt_eval)
    cn_traj = np.zeros(max_steps, dtype=np.float32)
    cp_traj = np.zeros(max_steps, dtype=np.float32)
    cn_dot_traj = np.zeros(max_steps, dtype=np.float32)
    cp_dot_traj = np.zeros(max_steps, dtype=np.float32)
```

```

cn_traj[0] = cn_true[idx, 0, 0]      # Initial un-normalized values for the
↪chosen trajectory
cp_traj[0] = cp_true[idx, 0, 0]

V_traj = np.zeros(max_steps, dtype=np.float32)
dG_dt_traj = np.zeros(max_steps, dtype=np.float32)
Z_traj = np.zeros((max_steps, 2), dtype=np.float32)
term1_traj = np.zeros((max_steps, 2), dtype=np.float32)
term2_traj = np.zeros((max_steps, 2), dtype=np.float32)
power_traj = np.zeros((max_steps), dtype=np.float32)

steps_run = 1
step = 0

# Rollout until depletion
while cn_traj[steps_run-1] >= cn_true[idx].min() and steps_run < max_steps:
    t_now = step * dt_eval

    x_t = torch.tensor([[
        transform(np.array([[[[I_t(t_now)]]]]), "I")[0,0,0], # True
        transform(np.array([[[[cn_traj[step]]]]], "cn")[0,0,0], #
↪Predicted
        transform(np.array([[[[cp_traj[step]]]]], "cp")[0,0,0]],
↪dtype=torch.float32).requires_grad_(True) # Predicted
↪# (1, 4)

    # Predict on initial state (1, 4)
    pred = model(x_t) # (1, 2)
    xi_dot_pred, V_pred, dG_dt, Z_pred, term1, power = state_evolver(pred,
↪x_t) # (1, 2)
    dc_n = inverse_transform(xi_dot_pred[0,0].detach().item(), 'cn_dot')
    dc_p = inverse_transform(xi_dot_pred[0,1].detach().item(), 'cp_dot')

    cn_traj[step+1] = cn_traj[step] + dc_n * dt_eval
    cp_traj[step+1] = cp_traj[step] + dc_p * dt_eval
    cn_dot_traj[step] = dc_n
    cp_dot_traj[step] = dc_p

    V_pred = inverse_transform(V_pred.detach().item(), 'V')
    V_traj[step] = V_pred
    dG_dt_traj[step] = dG_dt.detach().item()
    Z_traj[step] = Z_pred.detach().numpy()
    term1_traj[step] = term1.detach().numpy()
    power_traj[step] = power.detach().item()

    steps_run += 1
    step += 1

```

```

    # TODO: Trim to actual length ?
    return cn_traj, cp_traj, cn_dot_traj, cp_dot_traj, V_traj, dG_dt_traj,
    ↪Z_traj, term1_traj, power_traj, steps_run

```

```

[117]: # --- Plot rollout ---
def plot_rollout(cn_euler, cp_euler, cn_dot_euler, cp_dot_euler, steps_run,
                idx=0, dt_eval=DT, on_train=False):
    if on_train:
        cn_true = cn_train
        cp_true = cp_train
        cn_dot_true = cn_dot_train
        I_true = I_train
        dI_true = dI_train
    else:
        cn_true = cn_test
        cp_true = cp_test
        cn_dot_true = cn_dot_test
        I_true = I_test
        dI_true = dI_test

    # Ground truth valid length
    n_gt = int(mask_train_plot[idx].sum()) # number of valid GT steps
    n_nn = steps_run # number of NN steps before
    ↪stopping
    t_gt = np.arange(n_gt) * DT # physical time axis for GT
    t_nn = np.arange(n_nn) * dt_eval # physical time axis at new
    ↪resolution

    fig, axes = plt.subplots(1, 5, figsize=(20, 4))

    cn_err = (cn_true[idx, :n_nn, 0] - cn_euler[:n_nn]) / (cn_true[idx, :n_nn,
    ↪0])

    # --- cn err ---
    axes[0].bar(t_nn, cn_err, color="tab:blue")
    axes[0].set_xlabel("Time [s]")
    axes[0].set_ylabel("Relative error in $c_n$ [a.u.]")

    # --- cn ---
    axes[1].plot(t_gt, cn_true[idx, :n_gt, 0], color="tab:red",
    ↪linestyle="dashed", label="True $c_n$")
    axes[1].plot(t_nn, cn_euler[:n_nn], color="tab:blue", label="Predicted
    ↪$c_n$")
    axes[1].set_xlabel("Time [s]")

```



```

axes[1].set_ylabel(r"Concentration [mol/m$^3$]")
axes[1].legend()

# --- cp ---
axes[2].plot(t_gt, cp_true[idx, :n_gt, 0], color="tab:red",
↳linestyle="dashed", label="True $c_p$")
axes[2].plot(t_nn, cp_euler[:n_nn], color="tab:blue", label="Predicted_
↳$c_p$")
axes[2].set_xlabel("Time [s]")
axes[2].set_ylabel(r"Concentration [mol/m$^3$]")
axes[2].legend()

# --- cn_dot ---
axes[3].plot(t_gt, cn_dot_true[idx, :n_gt, 0], color="tab:red",
↳linestyle="dashed", label="True $\dot{c}_n$")
axes[3].plot(t_nn, cn_dot_euler[:n_nn], color="tab:blue", label="Predicted_
↳$\dot{c}_n$")
axes[3].set_xlabel("Time [s]")
axes[3].set_ylabel(r"Concentration rate [mol/m$^3$/s]")
axes[3].legend()

# --- current ---
axes[4].plot(t_gt, I_train[idx, :n_gt, 0], color="black", label="$I$")
axes[4].set_xlabel("Time [s]")
axes[4].set_ylabel("Current [A]")
axes[4].legend(loc='upper right')

# --- dI ---
axes3_ = axes[4].twinx()
axes3_.plot(t_gt, dI_train[idx, :n_gt, 0], color="tab:red",
↳linestyle="dashed", label="$\dot{I}$")
axes3_.set_ylabel(r"Current rate [A/s]")
axes3_.legend(loc='upper left')

plt.tight_layout()
# plt.savefig('figs/5.1_predict_ep50_GRF.pdf')
return t_gt, t_nn, n_gt, n_nn

```

```

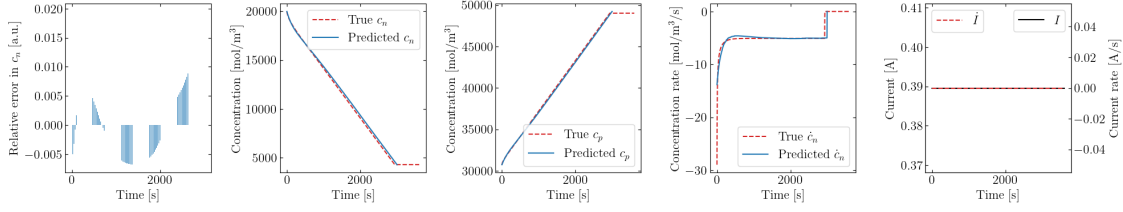
[118]: idx = 2
dt_eval = 3.7
on_train = False
cn_euler, cp_euler, cn_dot_euler, cp_dot_euler, V_traj, dG_dt_traj, Z_traj,
↳term1_traj, power_traj, steps_run = rollout(idx, dt_eval, on_train)

```

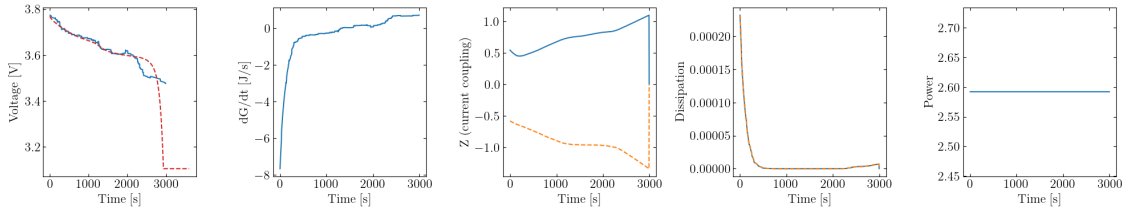
```

[119]: t_gt, t_nn, n_gt, n_nn = plot_rollout(cn_euler, cp_euler, cn_dot_euler,
↳cp_dot_euler, steps_run, idx, dt_eval, on_train)

```



```
[122]: # --- Plot from state evolver ---
fig, axes = plt.subplots(1, 5, figsize=(20, 4))
axes[0].plot(t_nn[:n_nn-1], V_traj[:n_nn-1]) # TODO: trim to actual
    ↪length
axes[0].plot(t_gt[:n_gt], V_test[idx, :n_gt, 0], color="tab:red",
    ↪linestyle="dashed", label="True $V$")
axes[0].set_xlabel("Time [s]")
axes[0].set_ylabel("Voltage [V]")
axes[1].plot(t_nn[:n_nn-1], dG_dt_traj[:n_nn-1])
axes[1].set_xlabel("Time [s]")
axes[1].set_ylabel("dG/dt [J/s]")
axes[2].plot(t_nn[:n_nn], Z_traj[:n_nn, 0], label="z1")
axes[2].plot(t_nn[:n_nn], Z_traj[:n_nn, 1], label="z2", linestyle='--')
axes[2].set_xlabel("Time [s]")
axes[2].set_ylabel("Z (current coupling)")
axes[3].plot(t_nn[:n_nn], term1_traj[:n_nn, 0], label="Dissipation")
axes[3].plot(t_nn[:n_nn], term1_traj[:n_nn, 1], label="Dissipation",
    ↪linestyle='--')
axes[3].set_xlabel("Time [s]")
axes[3].set_ylabel("Dissipation")
axes[4].plot(t_nn[:n_nn-1], power_traj[:n_nn-1], label="Power")
axes[4].set_xlabel("Time [s]")
axes[4].set_ylabel("Power")
fig.tight_layout()
```



```
[121]: # --- Reconstruct voltage ---
# Interpolate rollout concentrations onto the GT time grid
cn_euler_gt = np.interp(t_gt, t_nn, cn_euler[:n_nn])
```

```

cp_euler_gt = np.interp(t_gt, t_nn, cp_euler[:n_nn])

V_spm = SPM_eqs.V_terminal(I_test[idx, :n_gt, 0], cp_euler_gt, cn_euler_gt,
    ↪param)
V_true = V_test[idx, :n_gt, 0]

# I_nn      = np.interp(t_nn, t_gt, I_test[idx, :n_gt, 0])
# V_true_nn = np.interp(t_nn, t_gt, V_test[idx, :n_gt, 0])

# V_spm = SPM_eqs.V_terminal(I_nn, cp_euler[:n_nn], cn_euler[:n_nn], param)

ocv = SPM_eqs.OCV(cn_test[idx, :n_gt, 0], cp_test[idx, :n_gt, 0])

err = V_spm - V_true
err = err[np.isfinite(err)]

fig, axes = plt.subplots(1, 5, figsize=(20, 3.5))
axes[0].plot(t_gt, V_spm, label='SPM V')
axes[0].set_xlabel("Time [s]")
axes[0].set_ylabel("SPM Voltage [V]")
axes[0].plot(t_gt, V_true, label='True V', linestyle="dashed", color="tab:red")
axes[0].legend()
axes[1].hist(err, bins=100, label='True V')
axes[1].set_xlabel("Voltage error [V]")
axes[1].set_ylabel("Count [a.u.]")
axes[2].bar(t_gt, err / np.max(np.abs(V_true)), label='OCV')
axes[2].set_xlabel("Time [s]")
axes[2].set_ylabel("Relative Voltage Error [V]")
axes[3].plot(V_true - ocv, label='SPM V - OCV')
axes[3].set_xlabel("Time [s]")
axes[3].set_ylabel("Voltage Error [V]")
axes[4].plot(I_test[idx, :n_gt, 0], label='Current')

plt.tight_layout()
# plt.savefig('figs/5.1_SPM_V_ep100.pdf')

```

